

Parte 7 - Comunicazione tra Microservizi

Paradigmi di Comunicazione

Il passaggio da monolito a microservizi trasforma la comunicazione da intra-processo (chiamata di metodi diretti) a inter-processo (su rete, tramite protocolli come HTTP, AMQP, gRPC o TCP).

- **Sincrona (es. HTTP):** il client invia la richiesta e rimane in attesa (bloccante o non bloccante tramite `async/await`) finché non riceve la risposta.
 - *Svantaggi:* crea un forte accoppiamento e le catene di chiamate peggiorano drasticamente i tempi di risposta globali.
- **Asincrona (es. Kafka, RabbitMQ):** il client affida il messaggio a un *message broker* (intermediario) e prosegue la sua esecuzione senza attendere l'esito.
- **Destinatari:**
 - *Single Receiver:* la richiesta è elaborata da un solo destinatario (sincrono o asincrono).
 - *Multiple Receivers:* il messaggio è distribuito a più servizi (publish/subscribe, Event-Driven Architecture), fattibile solo in modalità asincrona.

Sovranità dei Dati e Replicazione

- L'obiettivo principale di un microservizio è l'autonomia: deve poter funzionare anche se gli altri servizi sono offline.
- **La Regola:** Evitare assolutamente chiamate HTTP sincrone per leggere dati appartenenti ad altri servizi.
- **La Soluzione:** Se un servizio (A) ha bisogno di una tabella "master" gestita da un servizio (B), i dati devono essere *replicati* asincronamente nel database del servizio A tramite messaggi.

Apache Kafka

Kafka è una piattaforma di data-streaming distribuita, basata su protocollo TCP, usata come message broker asincrono.

- **Componenti Chiave:**
 - *Topic:* Categoria logica a cui i messaggi vengono inviati.
 - *Partition:* Sottosezione di un topic; i record sono immutabili e vengono appesi in fondo in sequenza.
 - *Offset:* Indice univoco che identifica un record all'interno di una partizione.
 - *Producer/Consumer:* I client che, rispettivamente, pubblicano e leggono i messaggi.
 - *Cluster & Broker:* I server di Kafka, tipicamente coordinati dallo strumento *Apache ZooKeeper*.
- **Regole di Consumo (Consumer Groups):**
 - Ogni consumer appartiene a un *Group*. Un messaggio pubblicato su un topic arriva a *tutti* i gruppi iscritti, ma a *un solo* consumer all'interno dello stesso gruppo.
 - A ogni consumer viene associata dinamicamente una o più partizioni.

- Se ci sono più consumer che partizioni in un gruppo, alcuni consumer rimarranno inattivi senza ricevere dati.
- **Ordine dei Messaggi:** L'ordine di lettura (cronologico) è garantito rigorosamente **solo all'interno della singola partizione**.
- **Configurazione Offset:** Il parametro `auto.offset.reset` decide il comportamento all'avvio del consumer:
 - `earliest`: Legge l'intero storico dei messaggi.
 - `latest`: Ignora lo storico e legge solo i nuovi messaggi pubblicati da quel momento in poi.
- In *.NET*, si gestisce tramite la libreria NuGet `Confluent.Kafka`.

Il Pattern Transactional Outbox

Risolve il problema di mantenere allineato il database di un microservizio con i messaggi pubblicati su Kafka (evitando incoerenze in caso di crash).

- **Lato Producer:**
 1. Esegue una transazione sul proprio Database: aggiorna le proprie tabelle di business e, *contemporaneamente*, inserisce un record in una speciale tabella `TransactionalOutbox` (contenente il messaggio serializzato in JSON e il topic di destinazione).
 2. Un processo in background (`ProducerService`) legge periodicamente questa tabella, pubblica fisicamente il messaggio su Kafka e poi elimina il record.
- **Lato Consumer:**
 1. Un processo in background (`ConsumerService`) resta in attesa dei messaggi dal topic.
 2. Ricevuto l'evento, aggiorna il proprio database per riflettere le modifiche.

Comunicazione Sincrona (.NET)

Per le comunicazioni HTTP inter-servizio inevitabili, ASP.NET Core offre la classe `HttpClient`.

- **Typed Clients:** Consente di incapsulare l'uso di un API esterna in una classe specifica.
 - Si inietta `HttpClient` nel costruttore di questa classe custom.
 - Si registra nel `Program.cs` usando `builder.Services.AddHttpClient<IInterfaccia, ClasseImplementazione>(...)` dove è possibile configurare centralmente proprietà fisse, come il `BaseAddress` letto dal file di configurazione.

Condivisione Codice: Pacchetti NuGet Privati (GitHub)

Per condividere codice comune tra microservizi (es. modelli, interfacce), si usano pacchetti NuGet ospitati su *GitHub Packages Registry*.

- **Autenticazione:** Richiede la generazione di un PAT (Personal Access Token) su GitHub con permessi `read:packages` o `write:packages`.
- **NuGet.Config:** File XML da mettere a livello di soluzione (`.sln`), definisce i vari repository da interrogare e passa le credenziali lette dalle variabili d'ambiente.
- **Versioning SemVer 2.0 (`.csproj`):**
 - `<AssemblyVersion>`: Cambia *solo* se ci sono "breaking changes" (modifiche incompatibili col passato).

- `<PackageVersion>` : Formato `MAJOR.MINOR.PATCH` . `MAJOR` (breaking changes), `MINOR` (nuove feature retrocompatibili), `PATCH` (bugfix).

- **Comandi CLI:**

- `dotnet pack` : Crea il file d'archivio `.nupkg` .
- `dotnet nuget push` : Pubblica il pacchetto sul repository GitHub.